HopIn:University Carpooling Platform

Team Members: Jessica, Rida, Rudra, Suhayb, Youssef

Table of Contents

Introduction 3
Project Goals 3
Process and Exploration 4
System Design 4
  Architecture 4
  Domain Model 4
MVP Features and Implementation 5
  User Authentication 5
  Driver Ride Posting 5
  Ride Browsing and Searching 6
  Ride Request System 6
  Ride Confirmation 7
  Messaging 7
Project Management and Kanban Workflow 7
Feedback and Iteration 8
Testing and Validation 8
Technical Challenges and Solutions 9
Reflection 10
Future Improvements 10
Conclusion 10

Table of Figures

Figure 1: Login / Registration Page 5
Figure 2: Ride Posting Form 5
Figure 3: Ride Browsing Interface 6
Figure 4: Ride Request Interface 6
Figure 5: Ride Confirmation View 7
Figure 6: Messaging Interface 7
Figure 7: Kanban Board (Project Workflow) 8

Introduction

This project aimed to create HopIn, a web-based carpooling platform designed specifically for students and staff at the University of Regina. HopIn's goal is to provide a more affordable, convenient, and organized way for users to share rides to and from campus.

Commuting to university can be challenging due to rising transportation costs, limited public transit options, and lack of coordination between individuals traveling similar routes. HopIn addresses these challenges by connecting drivers with available seats to riders seeking reliable transportation within a trusted university-only environment.

The system was designed and developed following key software engineering principles, including Model-View-Controller (MVC) architecture, domain-driven design, and an Agile/Scrum-inspired process. This report outlines the team's exploration, design decisions, development process, and final outcomes.

Project Goals

The primary goal of this project was to design and develop a functional Minimum Viable Product (MVP) of a carpooling platform that demonstrates core ride-sharing functionality. The key objectives included:

- Developing a web-based application using MVC architecture
- Designing a clear and structured domain model
- Implementing essential ride-sharing features (posting, searching, requesting rides)
- Creating an intuitive and user-friendly interface
- Demonstrating effective teamwork and Agile project management

Additionally, the team aimed to build a system that solves a real-world problem while remaining feasible within a two-month development timeline.

Process and Exploration

At the beginning of the project, the team explored multiple potential ideas, including a skill-sharing platform and a scheduling system. While these ideas were viable, they lacked a strong real-world connection and clear user interaction flows. The team ultimately selected a carpooling platform because:

- It solves a common and relatable problem among students
- It has a well-defined domain (users, rides, requests)

- It fits naturally within MVC architecture
- It allows for meaningful user interaction

Throughout the planning phase, the team focused on defining a realistic MVP scope. Early ideas included additional features such as messaging payments and real-time tracking, but these were intentionally excluded to keep the project manageable and within our current technical skillset.

The development process followed an Agile-inspired approach, where tasks were broken into smaller units and tracked using a GitHub Kanban board. This allowed the team to monitor progress and adapt as needed.

System Design

Architecture

The HopIn system follows the Model-View-Controller (MVC) architecture:

- Model: Handles data and database interactions (users, rides, requests, ratings)
- View: Represents the user interface (forms, dashboards, listings)
- Controller: Manages logic, processes user actions, and connects the model to the view

This separation of concerns improves maintainability, scalability, and clarity of the system.

Domain Model

The system is built around the following key entities:

- User: Represents a student or staff member using the platform
- Ride: Represents a trip offered by a driver
- RideRequest: Represents a request made by a rider to join a ride
- Rating: Represents feedback provided after a completed ride
- Message: Enables communication between users

These entities form the foundation of the system and enable structured interactions between users.

MVP Features and Implementation

The team successfully implemented the following core features:

1. User Authentication
   Users can create accounts and log in securely. This ensures that only registered users can access the platform and interact with ride-sharing features.

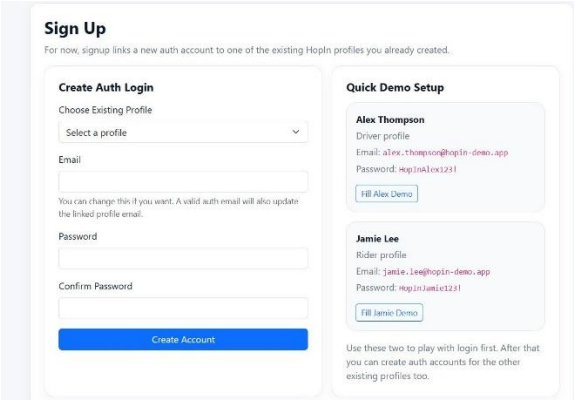

Figure 1

2. **Driver Ride Posting**
   Drivers can create ride listings by providing details such as destination, date, time, number of available seats, and pickup location.

   This feature enables drivers to offer rides to other users.

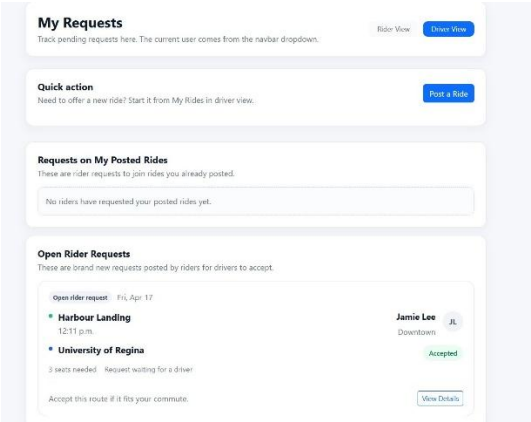

Figure 2

3. **Ride Browsing and Searching**
   Riders can browse available rides and filter them based on criteria such as time, location, and availability.

   This allows users to quickly find rides that match their needs.

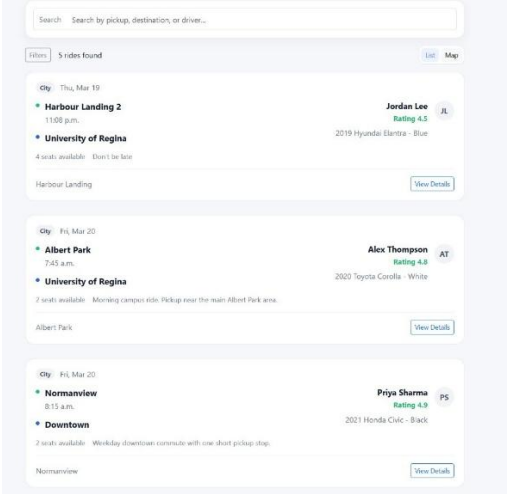

Figure 3

4. **Ride Request System**
   Riders can send requests to join a ride, and drivers can review and respond to these requests.
   This ensures controlled access to rides and prevents overbooking.

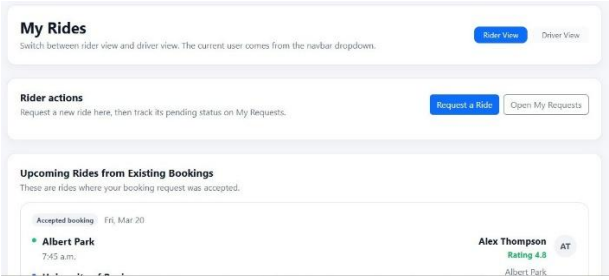

Figure 4

5. **Ride Confirmation**

   Drivers can accept or reject ride requests. Once accepted, the ride is confirmed and both users can proceed with the trip. This feature completes the core ride-sharing workflow.

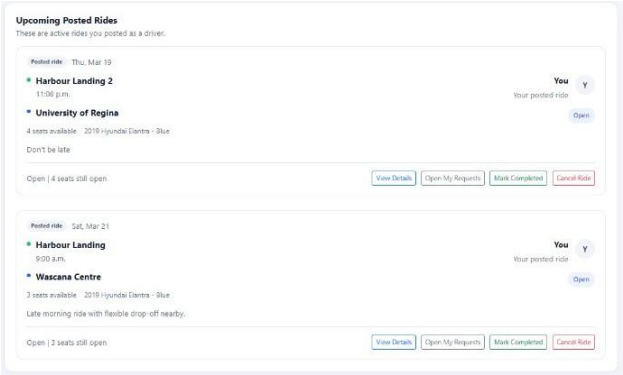

Figure 5

6. **Messaging**
   A basic messaging system allows drivers and riders to communicate regarding ride details, improving coordination and clarity.

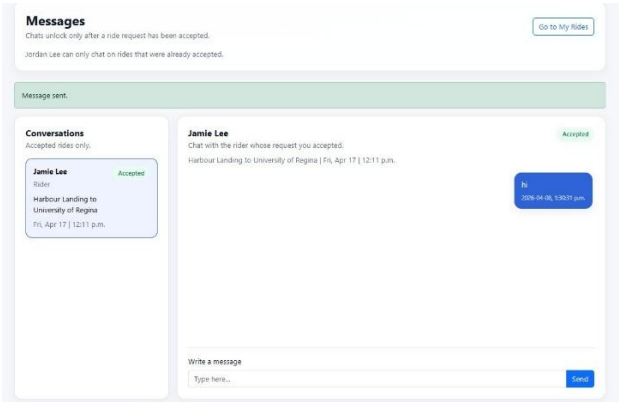

Figure 6

Project Management and Kanban Workflow

The team used a Kanban-based workflow to manage the development of the HopIn platform. Tasks were organized into three columns: To Do, In Progress, and Done, allowing the team to clearly track progress throughout the project.

Tasks included both project management activities (such as documentation and planning) and development tasks (such as implementing core features). Each task was assigned to team members to ensure accountability and balanced workload.

As the project progressed, tasks were moved across the board, providing a clear visual overview of the team's progress and helping identify any delays or issues. This approach supported an iterative development process, where the team focused first on core MVP features before refining and testing the system. Overall, the Kanban workflow improved organization, communication, and efficiency, contributing to the successful completion of the project.

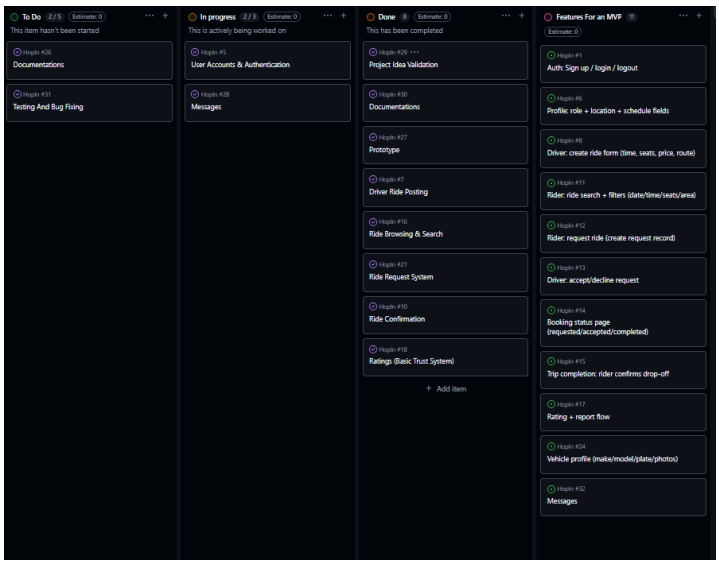

Figure 7

Feedback and Iteration

Throughout the project, the team received feedback from the instructor and peers. One key piece of feedback was to limit the scope of the MVP to ensure successful completion within the given timeframe. As a result, advanced features such as real-time tracking and payments were removed, the focus shifted to core ride-sharing functionality, and the user interface was simplified to improve usability. Additionally, feedback on the prototype led to improvements in layout and navigation, making the system more intuitive for users.

In future projects, the team would aim to gather feedback more proactively and continuously rather than at limited checkpoints. This could include conducting regular usability testing sessions, collecting survey responses, and analyzing user interactions with prototypes. By incorporating feedback earlier and more frequently, the team would be able to identify usability issues, refine features, and ensure that the final product better aligns with user expectations.

Testing and Validation

The team conducted testing on all major features of the application, including user authentication, ride posting, ride searching, and ride requests and confirmations. However, in future iterations, adopting a more structured and rigorous testing strategy would be beneficial. This would include implementing unit testing, where individual functions and components

(such as request handling logic or form validation) are tested in isolation to ensure correctness. For example, functions responsible for creating ride requests or updating ride status could be tested with various inputs to verify expected outputs.

In addition to unit testing, we would incorporate integration testing to verify that different parts of the system work together correctly. This is especially important in our project due to the interaction between the frontend, backend logic, and Supabase database. For instance, testing scenarios where a user submits a ride request to ensure that it is correctly stored in the database, reflected in the UI, and updated for both the driver and rider would be necessary and helpful for market deployment.

The team would also expand user testing by having real users interact with the system and provide feedback on usability, navigation, and overall experience. This could be done through prototype demonstrations or short surveys. Additionally, the team should test edge cases, such as invalid inputs, empty fields, or unexpected user behavior, to ensure the system remains stable under all conditions. Overall, a more comprehensive testing approach would significantly improve the reliability, usability, and quality of the system.

Technical Challenges and Solutions

During the development of the HopIn platform, the team encountered several technical challenges that required problem-solving and adaptation. One of the primary challenges was integrating the frontend interface with the backend database using Supabase. Since the course primarily focused on MongoDB, the team had to independently learn how Supabase handles authentication, data storage, and real-time updates. This created initial difficulties in understanding how data flows between the client and database, particularly when handling ride requests and updating ride statuses.

To address this, the team conducted additional research, tested different approaches, and used debugging techniques such as logging and step-by-step verification of data flow. By breaking down the integration into smaller components, the team was able to isolate issues and ensure that each part of the system functioned correctly before combining them.

Another challenge involved maintaining consistency across the user interface. As multiple team members worked on different pages, inconsistencies in layout, styling, and navigation emerged. This was resolved by standardizing components such as the navigation bar, layout structure, and styling conventions using shared CSS and reusable design patterns.

Additionally, managing asynchronous operations in JavaScript, such as handling database requests and updating the UI dynamically, introduced complexity. The team learned to properly use asynchronous functions and ensure that UI updates only occurred after data operations were successfully completed.

Overall, these challenges strengthened the team’s ability to troubleshoot, collaborate, and adapt to unfamiliar technologies. They also highlighted the importance of planning, consistency, and incremental testing when developing a full-stack application.

Reflection

This project provided valuable insights into both technical development and teamwork. From an individual perspective, team members learned the importance of balancing attention to detail with overall progress. While striving for high-quality work is important, focusing too much on minor details can slow down development. From a team perspective, communication was a key factor in success. The team learned that clear task distribution improves efficiency, regular check-ins help maintain alignment, and that flexibility and multitasking is necessary when challenges arise, especially for a small team. The project also reinforced the importance of Agile practices, as breaking work into manageable tasks made the development process more organized and less overwhelming. One repetitive challenge the team faced was related to coordinating different parts of the system, particularly integrating frontend and backend components especially because the team used Supabase for backend management instead of MongoDB which was taught in the lab. This was overcome through collaboration and some researching and testing.Overall, the experience highlighted that successful software development is not just about coding, but also about planning, communication, and adaptability.

Another important area for improvement identified during this project was the need for earlier and more continuous feedback and validation. While the team did receive feedback at some key stages, incorporating structured user and market feedback earlier in the design process would have allowed us to make more informed design decisions. Additionally, the team recognized that addressing code quality throughout development, rather than near the end, would have reduced the need for rework. Moving forward, the team would adopt a more iterative approach that combines continuous testing, feedback collection, and code refinement to improve both efficiency and overall product quality.

Future Improvements

In future iterations of the HopIn platform, several improvements could be implemented to enhance functionality and user experience. These include adding real-time ride tracking, integrating secure payment systems, improving the messaging system with instant notifications, and implementing stronger authentication and security measures. Additionally, expanding the platform to support a larger user base would require performance optimization and scalability considerations. These improvements would help transition the system from a functional MVP to a more complete and production-ready application.

Conclusion

The HopIn project successfully achieved its goal of developing a functional MVP for a university carpooling platform. The system demonstrates key software engineering principles, including MVC architecture, domain-driven design, and Agile project management. The final product provides a practical solution to a real-world problem, offering students and staff a cheaper and more efficient way to share rides. While there are

opportunities for future improvements, such as implementing real-time tracking, secure payment integration, enhanced messaging features, and improved security measures, these additions would require further technical development and testing. Future work would also focus on scalability, performance optimization, and refining the user experience based on real-world feedback. Overall, the current system provides a decent foundation that can be expanded into a fully functional public platform.